

Tema 3 – APIs e Integración de servicios y plataformas – Clase 3

EBIS Business Techschool

Contenidos

- Paginación, filtros y extracción masiva de datos
- Recorrer una API paginada
- Rate limits, reintentos y gestión robusta de errores
- Crear una función para llamar a una API robusta



Paginación, filtros y extracción masiva de datos



EBIS Business Techschool

El problema del volumen

EBIS Business Techschool

¿Qué pasaría si una API devolviera todos sus datos de golpe?

+400M

de repositorios públicos hay en GitHub

XXXX GB

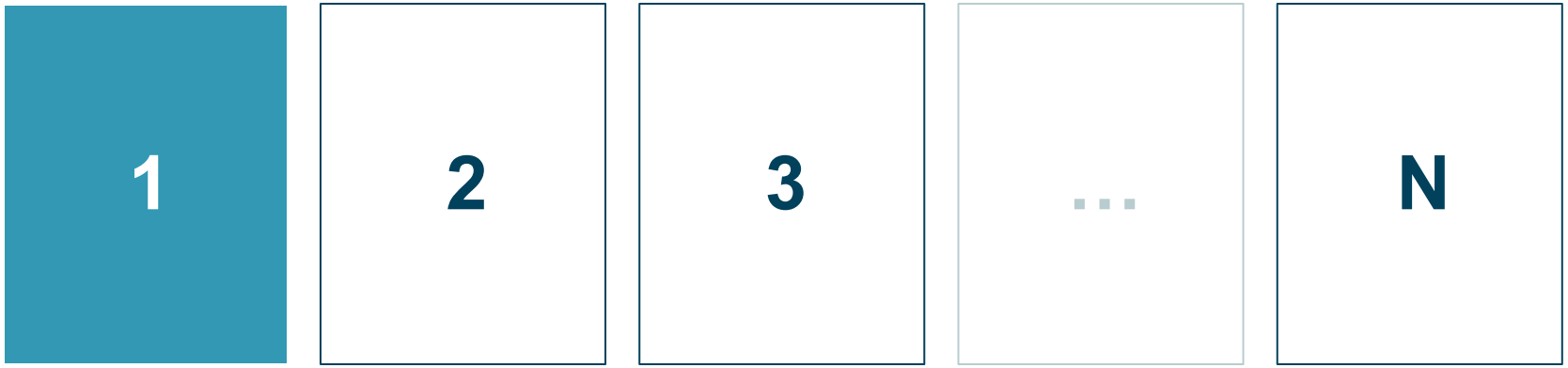
pesaría una sola respuesta con todo el listado

Timeout

la conexión se cortaría antes de terminar

Ninguna API seria devuelve colecciones grandes completas: las trocea.

Una petición, una página



- Cada petición devuelve un trozo (una página) de la colección
- Respuestas pequeñas: rápidas de generar, de enviar y de procesar
- Para obtenerlo todo, el cliente pide página tras página en un bucle

APIs de ejemplo

EBIS Business Techschool

API de GitHub

`api.github.com`

- Registro gratuito + token personal
- Paginación por página y por cursor
- Rate limits reales: 5.000 peticiones/hora

PokéAPI

`pokeapi.co`

- Sin registro
- Paginación por offset con enlace next
- Colección acotada: ~1.300 pokémon

Paginación por página y tamaño

El cliente pide una página concreta y un tamaño de página.

`page`

Qué página quiero: la 1, la 2, la 3…

`per_page`

Cuántos elementos por página (también page_size o limit)

- Como un buscador: resultados 1–50, 51–100, 101–150…
- Para avanzar, basta con sumar 1 al parámetro page

Ejemplo: page y per_page en GitHub

EBIS Business Techschool

```
GET api.github.com/users/octocat/repos ?per_page=50&page=2
```

- Devuelve los repositorios 51 a 100 del usuario
- per_page tiene un máximo: en GitHub, 100 elementos
- Una página más corta que per_page suele indicar que es la última

Paginación por offset

En lugar de páginas, se cuenta por posiciones: «sáltate N y dame los M siguientes».

`offset`

Cuántos elementos saltarse desde el principio

`limit`

Cuántos elementos devolver a partir de ahí



offset=200, limit=100: se salta 200 y devuelve del 201 al 300

Ejemplo: limit y offset en PokéAPI

EBIS Business Techschool

```
GET pokeapi.co/api/v2/pokemon?limit=100&offset=200
```

```
{ "count": 1302, "next ": "…/pokemon?offset=300&limit=100", "previous":
"…/pokemon?offset=100&limit=100", "results": [ … 100 elementos … ] }
```

La respuesta ya incluye la URL de la página siguiente.

Los límites del offset

EBIS Business Techschool

Datos que se mueven

Si se inserta o borra un elemento entre dos peticiones, las posiciones se desplazan: aparecen duplicados o se pierden registros.

Coste creciente

Para servir offset=1.000.000 el servidor recorre un millón de filas antes de devolver nada: cada página es más lenta que la anterior.

Por eso, para colecciones grandes o muy vivas existe un tercer modelo: el cursor.

Paginación por cursor

EBIS Business Techschool

Cada respuesta incluye un marcador (cursor) que apunta al punto exacto donde te quedaste.

- Es un valor opaco: no se calcula, no se interpreta, solo se reenvía
- Estable aunque los datos cambien entre peticiones: ni duplicados ni saltos
- Coste constante para el servidor: la página 10.000 tarda como la primera
- A cambio: no se puede saltar a una página concreta, solo avanzar

Nombres habituales: **`cursor`**, **`next`**, **`next_cursor`**, **`after`**, **`next_token`**

Ejemplo: el enlace next

EBIS Business Techschool

En el cuerpo de la respuesta (PokéAPI)

```
"next": "https://pokeapi.co/api/v2/pokemon?offset=300&limit=100"
```

En la cabecera Link (GitHub)

```
Link: <…repos?page=3>; rel="next", <…repos?page=8>; rel="last"
```

Regla práctica: sigue el enlace next hasta que deje de aparecer.

Comparativa de los tres tipos

EBIS Business Techschool

Tipo	Parámetros	Ventaja	Riesgo
Página y tamaño	`page, per_page`	Intuitivo, permite saltar a cualquier página	Inconsistente si los datos cambian
Offset	`offset, limit`	Control fino de la posición exacta	Duplicados y lentitud con offsets grandes
Cursor	`cursor, next, after`	Estable y rápido en colecciones enormes	Solo se avanza en secuencia

Para extracción masiva, el cursor es la opción más robusta.

Quiz 1

Una API devuelve en cada respuesta un campo «next» con un valor como «eyJpZCI6OTgx…». ¿Qué tipo de paginación usa?

A Paginación por página y tamaño

B Paginación por offset

C Paginación por cursor

EBIS Business Techschool

Filtrar en el servidor, no en local

EBIS Business Techschool

Sin filtro

Descargar 8.000 issues en 80 páginas y quedarse con las 240 que interesan.

80 peticiones

Con filtro en la petición

Pedir solo las issues que cumplen la condición y descargar únicamente esas.

3 peticiones

El dato más barato de mover es el que nunca se descarga.

Parámetros de filtro habituales en GitHub

EBIS Business Techschool

`state=open`

Filtrar por estado o categoría del recurso

`since=2026-01-01`

Solo elementos creados o modificados desde una fecha

`q=python`

Búsqueda por texto libre sobre la colección

`sort=updated&order=desc`

Ordenar los resultados antes de paginar

- Los nombres varían según la API: la documentación siempre manda

Filtros y paginación combinados

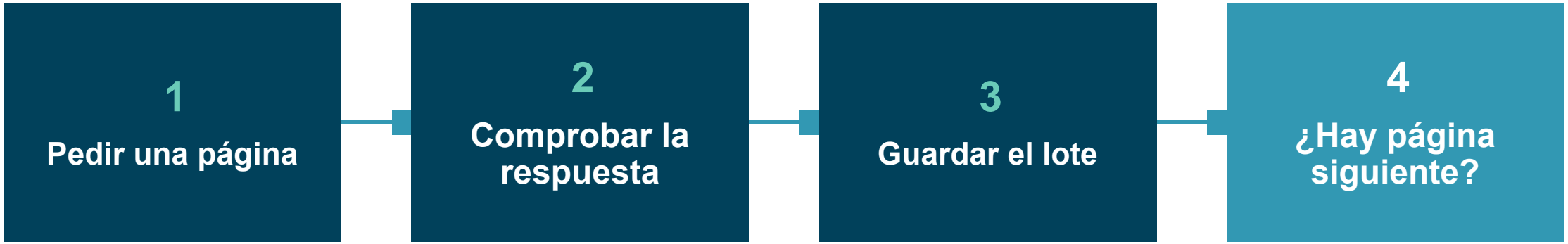
EBIS Business Techschool

```
GET api.github.com/repos/python/cpython/issues ?state=closed&since=2026-01-01 ← filtros
&sort=updated ← orden &per_page=100&page=1 ← paginación
```

- El servidor filtra y ordena primero; después trocea en páginas
- Si cambias un filtro, la numeración de páginas empieza de cero
- Mantén los mismos filtros durante toda la extracción

Anatomía del bucle de extracción

EBIS Business Techschool



← Sí: volver al paso 1 con la nueva página o cursor

No: fin de la extracción

Guardar va dentro del bucle, no al final: cada página se persiste al llegar.

Condiciones de parada

EBIS Business Techschool

¿Cuándo termina el bucle? Señales de que ya no hay más datos:

- El campo next llega vacío o desaparece de la respuesta
- La página llega vacía: cero resultados
- La página trae menos elementos que el limit pedido
- Se alcanza el total anunciado por el campo count

Además, siempre: un tope de seguridad (p. ej. máximo 1.000 páginas) por si la API o tu lógica fallan.

Errores comunes al recorrer páginas

EBIS Business Techschool

Bucle infinito

Olvidar actualizar page o el cursor: se pide la misma página para siempre.

Duplicados y huecos

Usar offset sobre datos que cambian mientras extraes.

Memoria desbordada

Acumular todas las páginas en una lista en vez de guardar por lotes.

Martillear la API

Encadenar peticiones sin pausa hasta agotar el rate limit.

Los cuatro se evitan con el diseño de bucle que acabamos de ver.

Extracción incremental

No vuelvas a descargarlo todo cada día: pide solo lo que ha cambiado.

Carga inicial (una vez)

Recorrer toda la colección página a página y guardarla completa.

Ejecuciones siguientes

Pedir solo lo modificado: `since=<última ejecución>`

- Guarda la fecha (o el último id) al terminar cada ejecución
- De horas de descarga a segundos: solo viajan las novedades

Persistencia intermedia

EBIS Business Techschool

Todo en memoria

El fallo en la página 900 de 1.000 se lleva por delante las 899 anteriores.

Guardado por lotes

Cada página se escribe al llegar (archivo o base de datos): lo descargado, descargado está.

- Un archivo por lote (datos_p001.json, datos_p002.json…) o inserciones por lote en BD
- La memoria se mantiene plana aunque la colección sea enorme

Checkpoints y reanudación

EBIS Business Techschool

Además de los datos, guarda «por dónde voy»: la última página o cursor completado.

checkpoint.json

```
{ "ultima_pagina": 480, "cursor": "eyJpZCI6OTgx…", "actualizado": "2026-07-03T10:41:00Z" }
```

- Se actualiza después de guardar cada lote, nunca antes
- Al arrancar, el script lee el checkpoint y continúa desde ahí
- Fallo a la página 480 de 600: se reanuda en la 481, no en la 1

Quiz 3

Tu extracción falla en la página 480 de 600. ¿Qué diseño te permite perder lo mínimo?

A Acumular todo en memoria y guardar al terminar

B Guardar cada lote al llegar y actualizar un checkpoint

C Guardar por lotes y tras el fallo consultar desde el inicio la integridad de los lotes

EBIS Business Techschool

Checklist de extracción segura

EBIS Business Techschool

- Identifica el tipo de paginación en la documentación
- Usa el tamaño de página máximo permitido
- Respeta el rate limit: pausas y backoff ante un 429
- Haz incrementales con since tras la carga inicial
- Filtra en el servidor antes de paginar
- Define condiciones de parada y un tope de seguridad
- Guarda por lotes y mantén un checkpoint actualizado
- Registra progreso y errores para poder auditar

Si cumples estos ocho puntos, tu extracción sobrevive a fallos, límites y datos cambiantes.

Resumen

EBIS Business Techschool

- Las APIs paginan para proteger al servidor, la red y tu memoria
- Tres modelos: página y tamaño, offset y cursor; para extraer en masa, cursor
- Filtra en el servidor y mantén los filtros estables durante la extracción
- Bucle seguro: condiciones de parada, pausas, lotes y checkpoint

Recorrer una API paginada



EBIS Business Techschool

Consultando APIs con paginación

Recorre los ~1.300 pokémon de PokéAPI siguiendo el enlace next, guardando un lote por página.

Rate limits, reintentos y gestión robusta de errores



EBIS Business Techschool

¿Por qué existen los límites de uso?

EBIS Business Techschool

Proteger

Un cliente descontrolado no puede tumbar el servicio para todos

Repartir

La capacidad se distribuye de forma justa entre miles de usuarios

Monetizar

Los planes de pago suelen ampliar el límite del plan gratuito

El límite no es un obstáculo: es parte del contrato de uso de la API.

Cómo se expresan los límites

EBIS Business Techschool

Por ventana de tiempo

«5.000 peticiones por hora», «100 por minuto»

Por identidad

Se cuenta por token, por cuenta o por dirección IP

Por endpoint

La búsqueda suele tener límites más estrictos que el resto

Por concurrencia

Máximo de peticiones simultáneas en vuelo

- Primer paso de toda integración: buscar «rate limit» en la documentación de la API

La API te avisa: cabeceras de rate limit

EBIS Business Techschool

```
HTTP/1.1 200 OK X-RateLimit-Limit: 5000 X-RateLimit-Remaining: 4870 X-RateLimit-Reset:
1783612800
```

- **`Limit`**: tu cupo total en la ventana actual
- **`Remaining`**: cuántas peticiones te quedan ahora mismo
- **`Reset`**: cuándo se renueva el cupo (timestamp)

Frenar antes de chocar

EBIS Business Techschool

Reactivo

Disparar sin mirar, chocar con el 429 y gestionar el bloqueo cuando ya ha ocurrido.

Proactivo

Leer Remaining en cada respuesta y, si se acerca a cero, esperar al Reset antes de seguir.

- Repartir el trabajo: mejor 5.000 peticiones a ritmo constante que en ráfaga
- Una pausa fija entre peticiones ya evita la mayoría de bloqueos

Control de volumen y rate limits

EBIS Business Techschool

60/hora

peticiones a GitHub sin autenticar

5.000/hora

peticiones con token personal gratuito

- La API avisa de tu margen en cabeceras: `X-RateLimit-Remaining`, `X-RateLimit-Reset`
- Pide el per_page máximo: 100 elementos por petición = 50 veces menos peticiones que de 2 en 2
- Añade una pequeña pausa entre peticiones: tu extracción no es urgente

La respuesta 429 y los reintentos

EBIS Business Techschool

429

Too Many Requests: «vas demasiado rápido»

- No es un error de tu código: es una orden de frenar
- Si llega la cabecera `Retry-After` , espera exactamente ese tiempo
- Si no, espera creciente: 1 s, 2 s, 4 s, 8 s… (backoff exponencial)
- Reintentar sin pausa solo alarga el castigo

Un extractor educado termina antes que uno impaciente.

La cabecera Retry-After


```
HTTP/1.1 429 Too Many Requests Retry-After: 120← «vuelve en 120 segundos»
```

- Puede venir en segundos (120) o como fecha exacta
- Si existe, se obedece exactamente: la API sabe cuándo te desbloquea
- Si no existe, aplicamos nuestra propia espera creciente (lo veremos en el apartado 3)

Errores recuperables y no recuperables

EBIS Business Techschool

Recuperables

`429 · 500 · 502 · 503 · 504 · timeout`

Fallos temporales: el tiempo los cura. Reintentar con espera tiene sentido.

No recuperables

`400 · 401 · 403 · 404 · 422`

Algo está mal en la petición. Reintentar sin cambiar nada es inútil: hay que corregir.

La pregunta clave ante todo error: ¿lo arregla el tiempo o lo tengo que arreglar yo?

¿Reintentar o no? Tabla de decisión

EBIS Business Techschool

Situación	¿Reintentar?	Acción
`429`	**Sí**	Esperar Retry-After (o backoff) y repetir
`5xx`	**Sí**	Backoff exponencial, con tope de intentos
`Timeout`	**Sí**	Reintentar con cuidado si la operación es segura de repetir
`401 / 403`	**No**	Revisar token y permisos; renovar credenciales
`400 / 404 / 422`	**No**	Corregir la petición: URL, parámetros o datos

¿Qué es un timeout?

El error sin código de estado: la respuesta, simplemente, no llega.

- Red caída, servidor colgado, petición perdida por el camino…
- Sin timeout, tu programa espera indefinidamente: se queda «colgado»
- El timeout es tú decidiendo cuánto estás dispuesto a esperar

Regla de oro: toda petición a una API lleva siempre un timeout definido.

Dos relojes: conexión y lectura

EBIS Business Techschool

Timeout de conexión

`orientativo: ~5 segundos`

Cuánto espero a que el servidor acepte la conexión. Si no responde rápido, probablemente esté caído.

Timeout de lectura

`orientativo: ~30 segundos`

Cuánto espero la respuesta una vez conectado. Consultas pesadas necesitan más margen.

- Un timeout que salta se trata como error recuperable: candidato a reintento

El reintento ingenuo empeora las cosas

EBIS Business Techschool

```
mientras falle: repetir petición inmediatamente← sin pausa, sin tope
```

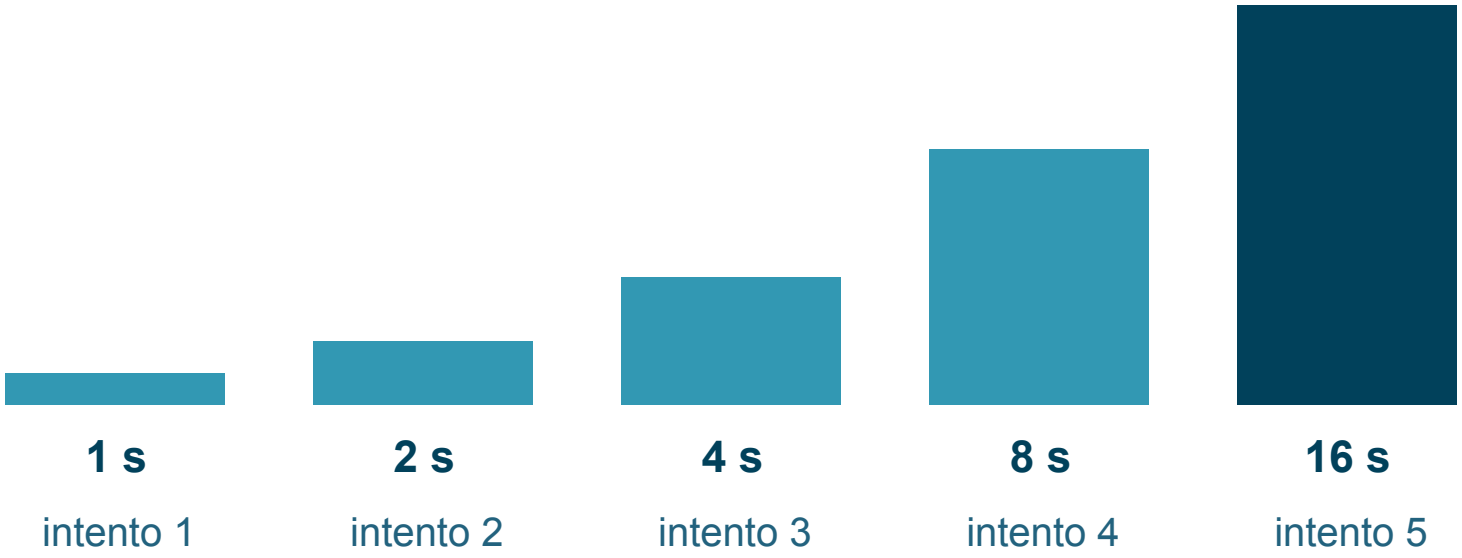
- A un servidor saturado le llegan ahora más peticiones que antes
- Miles de clientes reintentando a la vez: efecto estampida
- Quema tu rate limit en segundos y sin conseguir nada

Reintentar sí, pero con espera creciente y un número máximo de intentos.

Backoff exponencial

EBIS Business Techschool

Cada intento fallido duplica la espera antes del siguiente.



- Da al servidor tiempo real para recuperarse, sin abandonar la tarea

Jitter: un poco de azar en la espera

EBIS Business Techschool

Sin jitter

Mil clientes fallan a la vez, esperan exactamente 2 s… y vuelven a llegar todos a la vez. La ola se repite.

Con jitter

Cada cliente espera 2 s ± un margen aleatorio: los reintentos se reparten y el pico se disuelve.

Fórmula habitual: `espera = base × 2^intento + aleatorio`

Todo reintento necesita un tope

EBIS Business Techschool

- Habitual: 3 a 5 intentos como máximo por petición
- Si el servicio lleva caído una hora, reintentar no lo va a levantar
- Agotado el tope: registrar el fallo, guardar la tarea pendiente y avisar

La receta completa del reintento robusto

Solo errores recuperables + backoff exponencial + jitter + tope de intentos + respetar Retry-After.

¿Es seguro repetir? Idempotencia

EBIS Business Techschool

Tras un timeout no sabes si la petición llegó o no. ¿Qué pasa si la repites y la original sí llegó?

Seguras de repetir

Leer datos (GET), borrar por id (DELETE): repetirlas deja el mismo resultado.

Peligrosas de repetir

Crear un pedido, enviar un pago (POST): repetirlas puede duplicar la operación.

Solución habitual: una **clave de idempotencia** por operación; el servidor ignora la segunda llegada de la misma clave.

Quiz 2

Tu script recibe un 401 en cada petición desde hace 10 minutos. ¿Qué haces?

A Seguir reintentando con backoff

B Parar y revisar el token

C Esperar el Retry After para que se renueve el rate limit

Quiz 3

Una API te devuelve 503 de forma intermitente. ¿Cuál es la mejor estrategia?

A Reintentar sin pausa hasta que responda

B Abortar al primer 503: el servidor está caído

C Backoff exponencial con jitter

Logging básico: qué registrar

EBIS Business Techschool

Cuándo: fecha y hora de cada petición y respuesta

Qué: URL y método, sin el token ni secretos

Resultado: código de estado y tiempo de respuesta

Reintentos: número de intento y espera aplicada

```
2026-07-04 10:41:07 WARN GET /repos → 503 (intento 2/5, espero 4 s)
```

- Nunca registrar tokens, contraseñas ni datos personales

Niveles de log

`DEBUG`	Detalle fino para investigar: cada petición, cada cabecera
`INFO`	Hitos normales: «extracción iniciada», «página 40 guardada»
`WARN`	Algo raro pero manejado: un 429, un reintento, una respuesta lenta
`ERROR`	Fallo real que requiere atención: tope de reintentos agotado

En operación normal se vigilan WARN y ERROR; DEBUG solo al depurar.

Diseño defensivo: asumir que fallará

EBIS Business Techschool

Desconfía de la red

Toda petición puede fallar o no responder: timeout siempre.

Desconfía de la respuesta

Comprueba el código de estado y los campos antes de usarlos.

Falla con elegancia

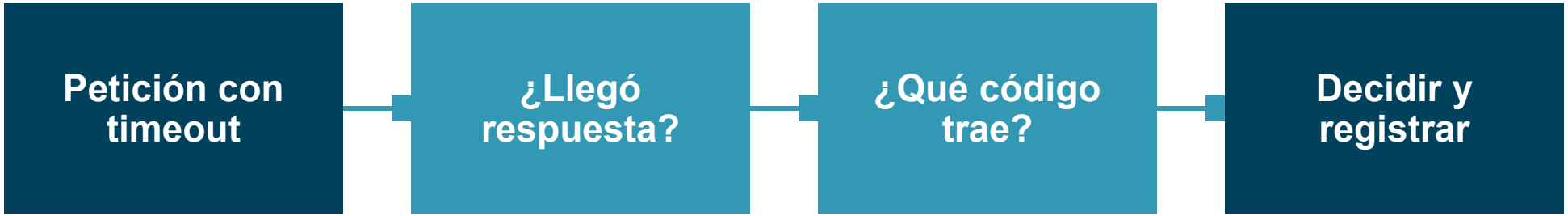
Un error no debe tirar todo el proceso: aislar, registrar, continuar.

Deja rastro

Logs suficientes para reconstruir qué pasó.

El código robusto no es el que nunca falla: es el que falla sin hacer daño.

El flujo completo de una petición robusta



- **2xx** → procesar los datos y continuar
- **429 / 5xx / timeout** → esperar (Retry-After o backoff + jitter) y reintentar, con tope
- **Otros 4xx** → no reintentar: registrar, corregir la causa
- **Tope agotado** → ERROR en el log, guardar la tarea pendiente, avisar

Checklist de integración robusta

EBIS Business Techschool

- Conoce el rate limit de la API antes de integrar
- Define timeout de conexión y de lectura en toda petición
- Reintenta solo lo recuperable, con backoff + jitter
- Comprueba si la operación es segura de repetir
- Lee las cabeceras X-RateLimit y frena a tiempo
- Clasifica cada error: ¿recuperable o no?
- Respeta Retry-After y pon tope de intentos
- Registra peticiones, errores y reintentos, sin secretos

Con estos ocho puntos, tu integración sobrevive a límites, caídas y redes inestables.

Resumen

EBIS Business Techschool

- El rate limit es parte del contrato: léelo en las cabeceras y frena antes del 429
- 4xx: corrige tu petición; 5xx y timeout: reintenta con cabeza
- Reintento robusto: backoff exponencial + jitter + tope + Retry-After
- Diseño defensivo: timeout siempre, fallar con elegancia y dejar rastro en los logs

Llamar de forma robusta a una API



EBIS Business Techschool

Consultando APIs con Rate Limit

Haz peticiones a la API de GitHub sin token hasta agotar las 60/hora y observa las cabeceras X-RateLimit y la respuesta de bloqueo.



EBIS Business Techschool

www.ebiseducation.com